

Mockito Hands-On Exercises

Exercise 1: Mocking and Stubbing

```
import static org.mockito.Mockito.*;
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;
import org.mockito.Mockito;

public class MyServiceTest {
    @Test
    public void testExternalApi() {
        ExternalApi mockApi = Mockito.mock(ExternalApi.class);
        when(mockApi.getData()).thenReturn("Mock Data");
        MyService service = new MyService(mockApi);
        String result = service.fetchData();
        assertEquals("Mock Data", result);
    }
}
```

Output:

```
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
```

Exercise 2: Verifying Interactions

```

import static org.mockito.Mockito.*;
import org.junit.jupiter.api.Test;
import org.mockito.Mockito;

public class VerifyInteractionTest {
    @Test
    public void testVerifyInteraction() {
        ExternalApi mockApi = Mockito.mock(ExternalApi.class);
        MyService service = new MyService(mockApi);
        service.fetchData();
        verify(mockApi).getData();
    }
}

```

Output:

```
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
```

Exercise 3: Argument Matching

```

import static org.mockito.Mockito.*;
import static org.mockito.ArgumentMatchers.*;
import org.junit.jupiter.api.Test;
import org.mockito.Mockito;

public class ArgumentMatchingTest {
    @Test
    public void testArgumentMatching() {
        ExternalApi mockApi = Mockito.mock(ExternalApi.class);
        MyService service = new MyService(mockApi);
        service.fetchDataById(101);
        verify(mockApi).getDataById(eq(101));
    }
}

```

Output:

```
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
```

Exercise 4: Handling Void Methods

```

import static org.mockito.Mockito.*;
import org.junit.jupiter.api.Test;
import org.mockito.Mockito;

public class VoidMethodTest {
    @Test
    public void testVoidMethod() {
        ExternalApi mockApi = Mockito.mock(ExternalApi.class);
        doNothing().when(mockApi).sendData("Test Data");
        MyService service = new MyService(mockApi);
        service.sendDataToApi("Test Data");
        verify(mockApi).sendData("Test Data");
    }
}

```

Output:

```
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
```

Exercise 5: Mocking and Stubbing with Multiple Returns

```

import static org.mockito.Mockito.*;
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;
import org.mockito.Mockito;

public class MultipleReturnsTest {
    @Test
    public void testMultipleReturns() {
        ExternalApi mockApi = Mockito.mock(ExternalApi.class);
        when(mockApi.getData())
            .thenReturn("First Call")
            .thenReturn("Second Call");
        MyService service = new MyService(mockApi);
        assertEquals("First Call", service.fetchData());
        assertEquals("Second Call", service.fetchData());
    }
}

```

Output:

```
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
```

Exercise 6: Verifying Interaction Order

```
import static org.mockito.Mockito.*;
import org.mockito.InOrder;
import org.junit.jupiter.api.Test;
import org.mockito.Mockito;

public class InteractionOrderTest {
    @Test
    public void testInteractionOrder() {
        ExternalApi mockApi = Mockito.mock(ExternalApi.class);
        MyService service = new MyService(mockApi);
        service.validateAndFetch();
        InOrder inOrder = inOrder(mockApi);
        inOrder.verify(mockApi).validate();
        inOrder.verify(mockApi).getData();
    }
}
```

Output:

```
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
```

Exercise 7: Handling Void Methods with Exceptions

```
import static org.mockito.Mockito.*;
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;
import org.mockito.Mockito;

public class VoidExceptionTest {
    @Test
    public void testVoidMethodThrowsException() {
        ExternalApi mockApi = Mockito.mock(ExternalApi.class);
        doThrow(new RuntimeException("Service Unavailable"))
            .when(mockApi).sendData("Test Data");
        MyService service = new MyService(mockApi);
        Exception exception = assertThrows(RuntimeException.class, () -> {
            service.sendDataToApi("Test Data");
        });
        assertEquals("Service Unavailable", exception.getMessage());
    }
}
```

Output:

Tests run: 1, Failures: 0, Errors: 0, Skipped: 0